

Prueba de Caja Blanca

“Sistema de Gestión de Caja para Tags y Productos”

Integrantes:

Martin Palacios

Matias Medina

Leopoldo Bolaños

Fecha: 18/06/2026

REQ003 CAJA DE PRODUCTOS:

Que de MTZ de HU

Es la Caja de productos para llevar los registros y movimientos de todos los productos

1. CÓDIGO FUENTE para Caja de productos

Pegar el trozo de código fuente que se requiere para el caso de prueba

```
package espe.edu.ec.programacaja.controller;

import espe.edu.ec.programacaja.model.CashFlowModel;
import espe.edu.ec.programacaja.model.SalesJournalModel;
import espe.edu.ec.programacaja.view.SalesPanel;

import javax.swing.*;
import java.awt.*;
import java.util.Map;

public class SalesController {

    private SalesPanel view;
    private SalesJournalModel model;
    private CashFlowModel cashFlowModel;

    public SalesController(SalesPanel view, CashFlowModel cashFlowModel) {
        this.view = view;
        this.model = new SalesJournalModel();
        this.cashFlowModel = cashFlowModel;

        installProductButtonsListeners();

        view.getBtnMas().addActionListener(e -> incrementarSeleccionado());
        view.getBtnMenos().addActionListener(e -> decrementarSeleccionado());

        view.getBtnGuardar().addActionListener(e -> guardarProductosEnCaja());

        actualizarSelector(null);
        refreshSummary();
    }

    private void installProductButtonsListeners() {
        Component leftPanel = view.getLeftPanel();

        if (leftPanel instanceof JPanel) {
            addListenerToAllButtons((JPanel) leftPanel);
        }
    }

    private void addListenerToAllButtons(JPanel panel) {
        for (Component comp : panel.getComponents()) {
            if (comp instanceof JPanel) {
                addListenerToAllButtons((JPanel) comp);
            }
        }
    }
}
```

```

    } else if (comp instanceof JButton) {
        JButton btn = (JButton) comp;
        String productName = btn.getActionCommand();

        if (productName != null && !productName.isEmpty()) {
            btn.addActionListener(e -> {
                model.addProduct(productName);
                refreshSummary();
                actualizarSelector(productName);
            });
        }
    }
}

private void incrementarSeleccionado() {
    String producto = view.getProductoSeleccionado();

    if (producto != null && !producto.isEmpty()) {
        model.addProduct(producto);
        refreshSummary();
        actualizarSelector(producto);
    }
}

private void decrementarSeleccionado() {
    String producto = view.getProductoSeleccionado();

    if (producto != null && !producto.isEmpty()) {
        model.removeProduct(producto);
        refreshSummary();

        if (model.getQuantities().containsKey(producto)) {
            actualizarSelector(producto);
        } else {
            actualizarSelector(null);
        }
    }
}

private void guardarProductosEnCaja() {
    if (!model.hasProducts()) {
        showError("No hay productos para guardar.");
        return;
    }

    try {
        Map<String, Integer> cantidades = model.getQuantities();
        Map<String, Double> precios = model.getPrices();

        for (Map.Entry<String, Integer> entry : cantidades.entrySet()) {

```

```

        String producto = entry.getKey();
        int cantidad = entry.getValue();
        double precio = precios.get(producto);

        cashFlowModel.addProductSale(producto, cantidad, precio);
    }

    model.clearProducts();

    refreshSummary();
    actualizarSelector(null);

    JOptionPane.showMessageDialog(
        view,
        "Productos guardados correctamente en el cierre de caja.",
        "Productos guardados",
        JOptionPane.INFORMATION_MESSAGE
    );

} catch (IllegalArgumentException ex) {
    showError(ex.getMessage());
}
}

private void actualizarSelector(String productoPreferido) {
    Map<String, Integer> cantidades = model.getQuantities();
    String[] productos = cantidades.keySet().toArray(new String[0]);
    view.actualizarListaProductos(productos, productoPreferido);
}

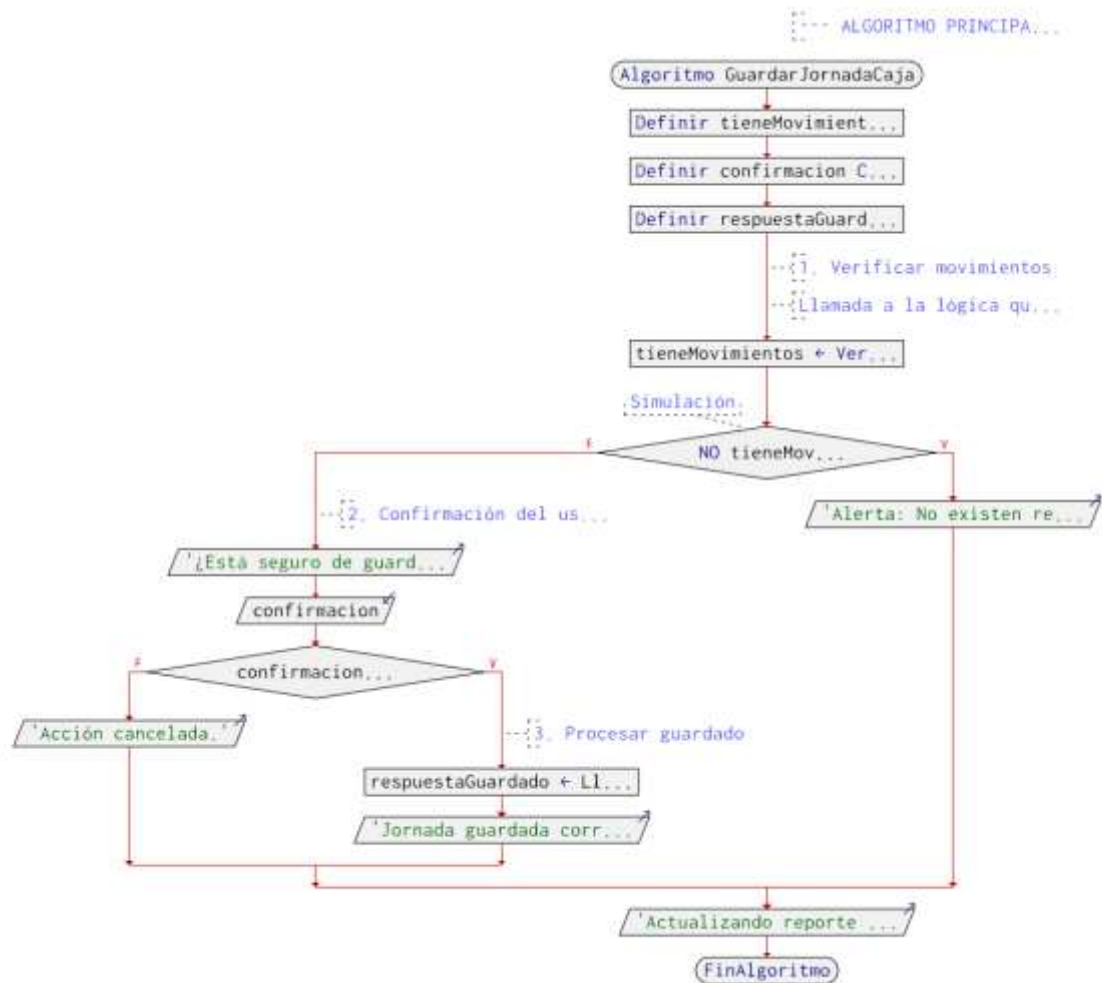
private void refreshSummary() {
    view.setResumenTexto(model.getFormattedReport());
}

private void showError(String message) {
    JOptionPane.showMessageDialog(
        view,
        message,
        "Error",
        JOptionPane.ERROR_MESSAGE
    );
}
}

```

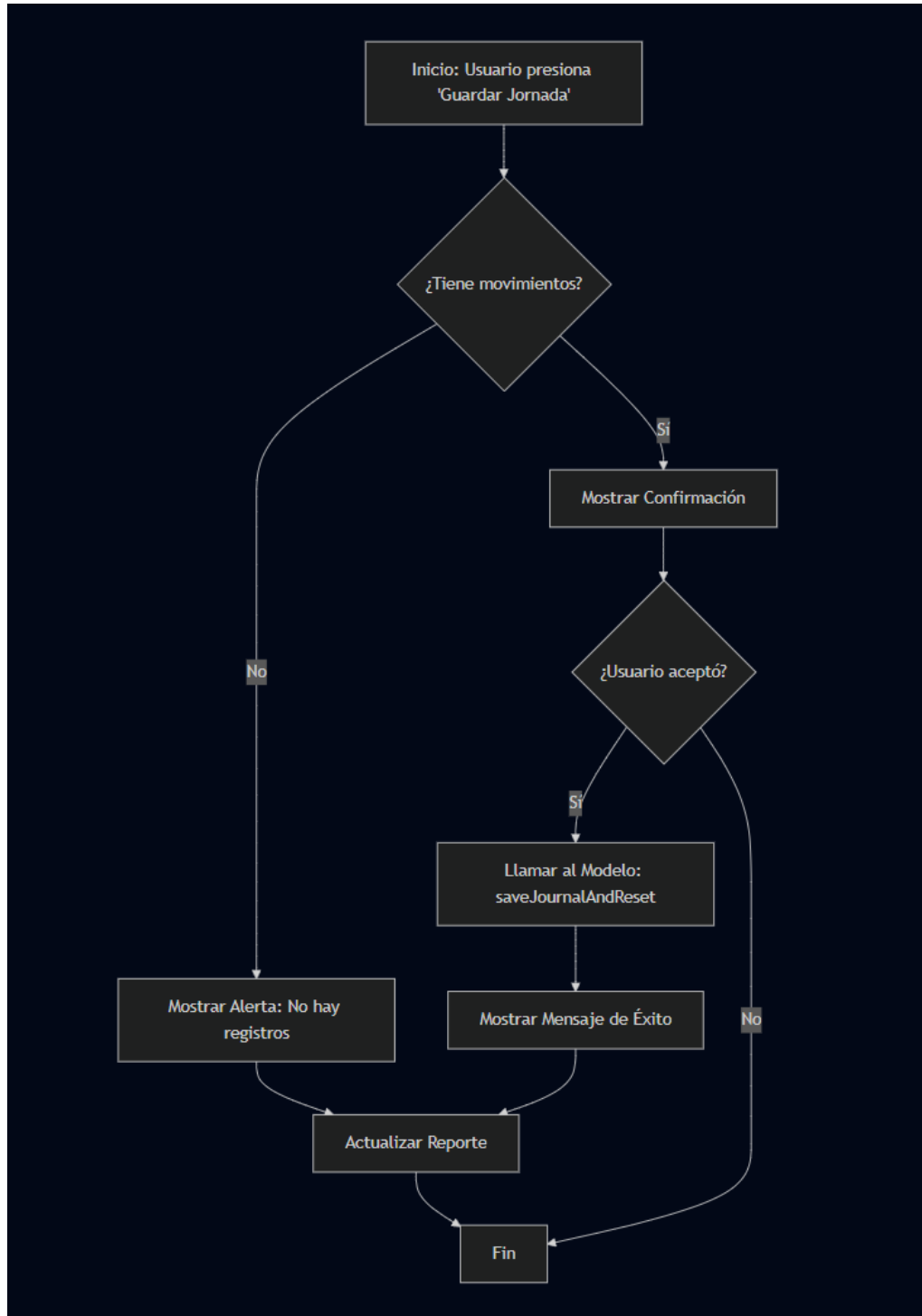
2. DIAGRAMA DE FLUJO (DF) Caja de Productos

Realizar un DF del código fuente del numeral 1



3. GRAFO DE FLUJO (GF) eliminación de contratos

Realizar un GF en base al DF del numeral



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) eliminación de contratos

R1 (Fallo: Jornada sin movimientos): 1 -> 2 -> 3 -> 7 -> 8

R2 (Fallo: Usuario cancela el cierre): 1 -> 2 -> 4 -> 5 -> 8

R3 (Éxito: Cierre de jornada guardado): 1 -> 2 -> 4 -> 5 -> 6 -> 7 -> 88

el mismo hasta el punto de decisión 7, pero la salida es diferente al no entrar al if success).

5. COMPLEJIDAD CICLOMÁTICA eliminación de contratos

6. A. Basada en nodos predichados: Fórmula: $V(G) = P + 1$ Cálculo: $V(G) = 2 + 1 = 3$

7. B. Basada en nodos y aristas (Fórmula de McCabe): Fórmula: $V(G) = A - N + 2P$ Cálculo: $V(G) = 9 - 8 + 2 = 3$

8. DONDE: P: 2 (Número de nodos predichado: 1. Validación de movimientos existentes, 2. Validación de confirmación del usuario). A: 9 (Número total de conexiones/flechas entre los bloques de ejecución). N: 8 (Número total de bloques o nodos en el flujo lógico).